

★ WEEK 13 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Price Patience · Fill the Q-Table · Beat the Naive Schedule

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — this week's environments are built inline; CSVs are hand-calc references

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Patience	reward streams on the slides — pure pencil work	— (two 3-step streams, no file needed)
P2 · Q-learning	toy walk built in-script (numpy only)	— (numbers on the slides)
P3 · Execution	impact + drift model defined in-script	fml_w13_p3_schedules.csv (hand-calc)
Extensions	same in-script simulators, bigger	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · PATIENCE

Price Patience With γ

Grab now vs invest-then-profit — one dial decides the winner

Two Strategies, Three Steps Each

Grab: (10, 0, 0). Invest: (-2, 0, 20). Score both with $G = r_1 + \gamma r_2 + \gamma^2 r_3$ at $\gamma = 0.5$ and $\gamma = 0.9$

Strategy	Step 1	Step 2	Step 3
Grab now	+10	0	0
Invest, then profit	-2	0	+20

Helper

- $0.5^2 = 0.25 \cdot 0.9^2 = 0.81$.
- The upfront -2 is never discounted — it happens now.

Your deliverables

- G for both strategies at both γ .
- The indifference γ^* (solve $-2 + 20\gamma^2 = 10$).
- One finance example where each γ is the right setting.

The Tool: the Discounted Return

Add every reward, but shrink the future by γ per step

$$G = r_1 + \gamma r_2 + \gamma^2 r_3 \quad \text{Grab: } (10, 0, 0) \quad \text{Invest: } (-2, 0, 20)$$

Reading it

- γ multiplies once per step of delay: step 3 carries γ^2 .
- γ near 0: only now matters. γ near 1: the future counts almost fully.
- The agent maximizes G — so γ silently defines its personality.

Lab guide — per γ

- Step 1: Grab's G is just 10 (nothing to discount).
- Step 2: Invest's $G = -2 + \gamma^2 \cdot 20$ (helpers given).
- Step 3: compare; then set the two equal and solve for γ^* .

Two Discounts and One Threshold

The same sacrifice is a blunder at $\gamma = 0.5$ and a bargain at $\gamma = 0.9$

γ	Grab: G	Invest: $G = -2 + \gamma^2 \cdot 20$	Winner
0.5	10	$-2 + 0.25 \cdot 20 = 3.0$	GRAB by 7
0.9	10	$-2 + 0.81 \cdot 20 = 14.2$	INVEST by 4.2
$\gamma^* : -2 + 20\gamma^2 = 10$	$\rightarrow \gamma^2 = 0.6$	$\gamma^* = \sqrt{0.6} \approx 0.775$	the indifference point

$$\gamma=0.5: 10 \text{ vs } 3 \text{ (grab)}$$

$$\gamma=0.9: 10 \text{ vs } 14.2 \text{ (invest)}$$

$$\gamma^* = \sqrt{0.6} \approx 0.775$$

γ Is the Agent's Time Preference

Below 0.775 the future is too cheap to invest in; above, sacrifice pays

3.0

Invest's score at $\gamma = 0.5$ — patience priced at half decays fast

14.2

Invest's score at $\gamma = 0.9$ — the future nearly at face value

0.775

The exact γ where the two strategies tie ($\sqrt{0.6}$)

γ^2

Why: two steps of delay compound the discount

Finance settings

- Intraday execution: horizon minutes $\rightarrow$ modest γ is honest.
- Learning a client relationship: sacrifices repay for years $\rightarrow \gamma$ high.

The quiet trap

- A γ mismatched to the task trains the wrong personality.
- Greedy agents look great early and cap out; patient ones invert.

Scale It: Watch γ Reshape Behavior

Paste the metaprompt — Claude Code sweeps the dial and shows the personality change

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) verifies the toy first: streams (10,0,0) and (-2,0,20) $\rightarrow$ G at gamma 0.5 (10 vs 3) and 0.9 (10 vs 14.2); solve and print $\gamma^* = \sqrt{0.6}$;
- 2) plots both $G(\gamma)$ lines for γ in [0, 1] and marks the crossing at 0.775;
- 3) builds a 10-step toy: at each step the agent may 'harvest' +1 or 'plant' -0.5 that pays +3 two steps later; Q-learning with γ in [0.3, 0.775, 0.95], 500 episodes each;
- 4) prints, per γ , the learned plant-rate and the average total (undiscounted) reward achieved;
- 5) three-sentence verdict: how γ changed the PERSONALITY, in this week's language.

[OUTPUT] What to expect

toy check: 3 / 14.2 and
 $\gamma^* = 0.7746$
crossing plot at 0.775
 γ 0.3: never plants
 γ 0.95: plants often,
highest total reward
0.775: on the fence

Your one-dial threshold,
grown into a personality.

P2

PROBLEM 2 · Q-LEARNING

Fill the Table by Hand

Three episodes on a three-state walk — watch the goal's glow spread backward

A Three-State Walk to a Reward

States $0 \rightarrow 1 \rightarrow 2$ (goal, reward 1). $\alpha = 0.5$, $\gamma = 0.9$, all Q start at 0. The agent goes right each time — update Q for three episodes

Update rule per step: $Q(s, R) \leftarrow Q(s, R) + 0.5 \times [r + 0.9 \cdot \max Q(\text{next}) - Q(s, R)]$.

Episode = two steps: $0 \rightarrow 1$ ($r = 0$), then $1 \rightarrow 2$ ($r = 1$, done).

Given

- Reaching state 2 pays $r = 1$; every other step pays 0.
- $\max Q(2, \cdot) = 0$ forever (the episode ends there).
- Work in exact decimals — they stay clean.

Your deliverables

- $Q(0, R)$ and $Q(1, R)$ after episodes 1, 2, 3.
- The episode at which $Q(0, R)$ first becomes positive — and why.
- One sentence naming what 'spreads backward'.

The Tool: Nudge Toward Reward-Plus-Best-Next

Each update is a small correction toward what actually happened

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad \alpha=0.5, \gamma=0.9$$

Reading it

- The bracket is the SURPRISE: what you saw minus what you believed.
- $\alpha = 0.5$ means: move your belief halfway toward the surprise.
- $\gamma \cdot \max Q(\text{next})$ is how tomorrow's promise reaches today's cell.

Lab guide — per episode

- Step A: update $Q(0, R)$ using $\max Q$ at state 1.
- Step B: update $Q(1, R)$ using $r = 1$ at the goal.
- Note the order: state 0 updates BEFORE state 1's new value exists.

Six Updates, Two Columns

The goal's glow reaches state 1 in episode 1 — and state 0 only in episode 2

Episode	$Q(0, R)$ update	$Q(1, R)$ update
1	$0 + 0.5 \cdot (0 + 0.9 \cdot 0 - 0) = 0$	$0 + 0.5 \cdot (1 - 0) = 0.5$
2	$0 + 0.5 \cdot (0.9 \cdot 0.5 - 0) = 0.225$	$0.5 + 0.5 \cdot (1 - 0.5) = 0.75$
3	$0.225 + 0.5 \cdot (0.9 \cdot 0.75 - 0.225) = 0.45$	$0.75 + 0.5 \cdot (1 - 0.75) = 0.875$

$Q(1, R): 0 \rightarrow 0.5 \rightarrow 0.75 \rightarrow 0.875$ $Q(0, R): 0 \rightarrow 0 \rightarrow 0.225 \rightarrow 0.45$ (the goal's glow spreads backward)

Temporal-Difference Learning, Witnessed

Value flows backward from the goal, one state per episode, at the speed of α

0.5

$Q(1,R)$ after one episode — the goal's neighbor learns first

ep. 2

When $Q(0,R)$ turns positive — only via state 1's new value

0.9

The target $Q(1,R)$ crawls toward $(\gamma \cdot 1)$ caps... at $r=1$ it's 1)

halfway

What $\alpha = 0.5$ does to every surprise, every step

Why the order mattered

- State 0 updated against the OLD $Q(1)$ each episode.
- That one-step lag is why the glow spreads one hop per episode.

Scaling up

- Six states (Lab 1) — same mechanics, longer corridor.
- Replace the table with a W8 network and you have deep RL.

Scale It: From Three States to a Learning Curve

Paste the metaprompt — Claude Code verifies your table and grows the corridor

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: 3-state walk, alpha=0.5,  
   gamma=0.9, deterministic right moves, 3 episodes ->  
   print Q after each; expect Q(1,R): .5/.75/.875 and  
   Q(0,R): 0/.225/.45;  
2) runs the full 6-state Lab-1 world with epsilon-  
   greedy (eps=0.2) for 200 episodes; plot steps-to-  
   goal per episode (the learning curve);  
3) heatmap the final Q-table; confirm 'right' wins in  
   every state;  
4) rerun with alpha in [0.05, 0.5, 0.95] and overlay  
   the three learning curves;  
5) three-sentence verdict: what each alpha did to  
   stability vs speed, in this week's language.
```

[OUTPUT] What to expect

```
toy check: your six  
           updates, digit for digit  
learning curve: falls from  
               wandering to ~5 steps  
heatmap: right column  
        bright, decaying with  
        distance from the goal  
alpha 0.95: fast, jittery  
alpha 0.05: smooth, slow  
  
Your six updates, run  
40,000 more times.
```

P3

PROBLEM 3 · EXECUTION

Three Schedules, One Bill

Impact vs drift — price them by hand, then beat TWAP

Buy 10,000 Shares in a Rising Market

Impact $k \cdot t^2$ with $k = 0.00002$; drift 0.01 per period over 10 periods. Price three schedules

Schedule	The trades $t_1 \dots t_{10}$
All at once	10,000, then nothing
TWAP	1,000 every period
Front-half	2,000 in periods 0–4, then nothing

Helper

- $k \cdot t^2$: $10,000^2 = 10^8 \cdot 2,000^2 = 4 \cdot 10^6 \cdot 1,000^2 = 10^6$.
- Drift in period i : $t \cdot i \cdot 0.01$ (period 0 free).

Your deliverables

- Impact, drift, and total for all three schedules.
- The winner — and the condition that would flip it.

The Tool: Two Costs Pulling Opposite Ways

Impact punishes speed; drift punishes patience — the schedule is a treaty between them

$$\text{Cost} = \underbrace{\sum_i k t_i^2}_{\text{impact}} + \underbrace{\sum_i t_i \cdot i \cdot \text{drift}}_{\text{waiting in a rising market}} \quad k = 0.00002, \text{ drift} = 0.01$$

Reading it

- The t^2 makes impact QUADRATIC: one 10,000 costs $100 \times$ a 1,000.
- Drift is linear in size but grows with the wait ($\times i$).
- Slicing crushes the square; tilting early ducks the drift.

Lab guide — per schedule

- Step 1: impact = $\sum k \cdot t^2$ (helpers give the squares).
- Step 2: drift = $\sum t \cdot i \cdot 0.01$.
- Step 3: add, compare, and name the winner's trade-off.

Three Bills, Itemized

Slicing crushed the square; the early tilt then ducked the drift

Schedule	Impact $\Sigma k \cdot t^2$	Drift $\Sigma t \cdot i \cdot 0.01$	Total
All at once	$0.00002 \cdot 10^8 = 2,000$	0	2,000
TWAP (1,000 $\times$ 10)	$10 \cdot 0.00002 \cdot 10^6 = 200$	$1,000 \cdot (0 + \dots + 0.09) = 450$	650
Front-half (2,000 $\times$ 5)	$5 \cdot 0.00002 \cdot 4 \cdot 10^6 = 400$	$2,000 \cdot (0 + \dots + 0.04) = 200$	600

all-at-once: 2,000

TWAP: 650

front-loaded: 600 ✓

600 Beats 650 Beats 2,000

Flip the drift negative and TWAP retakes the lead: there is no universal schedule

-70%

TWAP's saving over the slam — discipline alone

+200

Impact the front-tilt paid extra (400 vs 200)

-250

Drift it dodged in exchange (200 vs 450) — net -50

per day

The right tilt changes with conditions — the agent's job

The RL framing

- State (shares, time, price) → action (slice size) → reward (-cost).
- The agent rediscovers this treaty — and adjusts it live.

The honesty rider

- The winner is winner ONLY under this k and this drift.
- W10 applies to agents too: purge, deflate, distrust pretty sims.

Scale It: Let an Agent Find the Treaty Itself

Paste the metaprompt — Claude Code grid-searches, then trains a Q-learner on the schedule

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) verifies the toy first: $k=0.00002$, $\text{drift}=0.01$, 10,000 shares, 10 periods $\rightarrow$ confirm totals 2,000 / 650 / 600; (reference: `fml_w13_p3_schedules.csv`)
- 2) grid-search: over front-load fractions 0..100% (the rest spread evenly), plot total cost vs fraction and mark the minimum;
- 3) repeat for drift in $[-0.01, 0, +0.01]$ and overlay the three curves - show the optimum moving;
- 4) Q-learning agent: state = (periods left, shares left bucketed), action = slice in $\{0, 1000, 2000, 3000\}$; train 5,000 episodes at drift $+0.01$; print its learned schedule and cost vs TWAP;
- 5) three-sentence verdict: what treaty the agent found, in this week's language.

[OUTPUT] What to expect

toy check: 2,000/650/600
exactly
cost curve: U-shape with a
minimum near mild front-
loading
negative drift: optimum
slides to back-loading
agent: front-tilted slices,
cost at or below 600

Your three bills, renegoti-
ated 5,000 times.

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

$\gamma^* = \sqrt{0.6} \approx 0.775$ — patience has a price

The same sacrifice scored 3 at $\gamma = 0.5$ and 14.2 at $\gamma = 0.9$. The dial is the agent's personality.

2

Q spreads backward: $0.5 \rightarrow 0.75 \rightarrow 0.875$ and $0 \rightarrow 0.225 \rightarrow 0.45$

Six updates showed temporal-difference learning: the goal's glow travels one state per episode, at speed α .

3

Execution treaty: $2,000 \rightarrow 650 \rightarrow 600$

Slicing crushed the quadratic; a front tilt then dodged the drift — and the right tilt changes daily, which is the agent's job.

Hand calculation first — then let Claude Code scale the same idea to real data.